

WEEK-6 SKILL

1 . To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

Aim : To create and test process escape sequences in Ubuntu, handle escaped spaces, escape special symbols, preserve characters, validate parser output, and test complex inputs.

Program :

```
GNU nano 7.2
#include <stdio.h>

int main()
{
    printf("Process Escape Sequence Demonstration\n\n");

    printf("1. Normal text:\n");
    printf("Hello World\n\n");

    printf("2. Escaped space:\n");
    printf("Hello\\ World\n\n");

    printf("3. Escaped dollar symbol:\n");
    printf("\\$HOME\n\n");

    printf("4. Escaped star symbol:\n");
    printf("\\*\\n\n");

    printf("5. Escaped question mark:\n");
    printf("\\?\\n\n");

    printf("6. Escaped semicolon:\n");
    printf("Hello\\;World\n\n");

    printf("7. Escaped pipe symbol:\n");
    printf("Hello\\|World\n\n");

    printf("8. Escaped ampersand:\n");
    printf("Hello\\&World\n\n");

    printf("9. Complex input:\n");
    printf("Hello\\ World\\ \\$USER\\ \\*\\ \\?\\ \\&\\ \\;\\ \\/\\n\n");

    printf("10. Quoted text:\n");
    printf("'Hello $USER * ? & ; |'\n");

    return 0;
}
```

```
GNU nano 7.2
#!/bin/bash

echo "==== PROCESS ESCAPE SEQUENCE TEST ====="

echo "Test 1: Escaped Space"
echo Hello\ World

echo "Test 2: Escaped Dollar"
echo \$HOME

echo "Test 3: Escaped Star"
echo \*

echo "Test 4: Escaped Question Mark"
echo \?

echo "Test 5: Escaped Semicolon"
echo Hello\;World

echo "Test 6: Escaped Pipe"
echo Hello\|World

echo "Test 7: Escaped Ampersand"
echo Hello\&World

echo "Test 8: Single Quotes"
echo '$HOME * ? & ; |'

echo "Test 9: Double Quotes"
echo "$HOME"

echo "Test 10: Complex Input"
echo Hello\ World\ \ $USER\ \*\ \?\ \&\ \;\ \|

echo "==== TEST COMPLETED ====="
```

Output :

```

'Hello $USER * ? & ; |'
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo Hello\ World
Hello World
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo \$HOME
$HOME
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo $HOME
/home/jhanasri
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo \*
*
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo \?
?
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo Hello\;World
Hello;World
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo Hello\|World
Hello|World
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo Hello\&World
Hello&World
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo '$HOME'
$HOME
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo 'Hello $USER * ? & ; |'
Hello $USER * ? & ; |
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo "Hello World"
Hello World
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo "Hello $USER"
Hello jhanasri
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo "Hello $USER"
Hello jhanasri
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo '$HOME'
$HOME
jhanasri@LAPTOP-1RN1M8E3:~/week6$ echo Hello\ World\ \$USER\ \*\ \?\ \&\ \;\ \|
Hello World $USER * ? & ; |
jhanasri@LAPTOP-1RN1M8E3:~/week6$ nano parser_test.sh
jhanasri@LAPTOP-1RN1M8E3:~/week6$ chmod +x parser_test.sh
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ls -l parser_test.sh
-rwxr-xr-x 1 jhanasri jhanasri 594 Sep 18 13:33 parser_test.sh
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ./parser_test.sh

==== PROCESS ESCAPE SEQUENCE TEST ====
Test 1: Escaped Space
Hello World
Test 2: Escaped Dollar
$HOME
Test 3: Escaped Star

```

```

*
Test 4: Escaped Question Mark
?
Test 5: Escaped Semicolon
Hello;World
Test 6: Escaped Pipe
Hello|World
Test 7: Escaped Ampersand
Hello&World
Test 8: Single Quotes
$HOME * ? & ; |
Test 9: Double Quotes
/home/jhanasri
Test 10: Complex Input
Hello World $USER * ? & ; |
===== TEST COMPLETED =====
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ./parser_test.sh | grep "Hello World"
Hello World
Hello World $USER * ? & ; |
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ./parser_test.sh | grep '\$HOME'
$HOME
$HOME * ? & ; |
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ./parser_test.sh | grep '\*'
*
$HOME * ? & ; |
Hello World $USER * ? & ; |
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ./parser_test.sh | grep '?'
?
$HOME * ? & ; |
Hello World $USER * ? & ; |
jhanasri@LAPTOP-1RN1M8E3:~/week6$ touch my\ file.txt
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ls
escape escape.c 'my file.txt' parser_test.sh
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ls my\ file.txt
'my file.txt'
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ls "my file.txt"
'my file.txt'
jhanasri@LAPTOP-1RN1M8E3:~/week6$ touch 'student file.txt'
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ls
escape escape.c 'my file.txt' parser_test.sh 'student file.txt'
jhanasri@LAPTOP-1RN1M8E3:~/week6$ ls 'student file.txt'
'student file.txt'

```

Result:

The process escape sequence program was successfully implemented and executed in Ubuntu. Escaped spaces and special characters such as \$, *, ?, ;, |, and & were handled correctly. Single quotes, double quotes, and complex inputs were also tested successfully.

Observation:

The program correctly preserved escaped characters and spaces. Special symbols were handled using escape sequences, while variable expansion worked correctly with double quotes. All the tested inputs produced the expected output.

2 . To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching.

Aim : To create a child process using `fork()`, execute another program using `exec()`, handle execution errors, pass arguments to the executed program, manage the parent process, and test command launching in Ubuntu.

Program :

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        // Child process
        printf("\nChild process created successfully.\n");
        printf("Child PID: %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting ls command using execlp()...\n");

        execlp("ls", "ls", "-l", NULL);

        // This executes only if execlp() fails
        perror("execlp failed");
        exit(1);
    }
    else
    {
        // Parent process
        printf("\nParent is waiting for child process...\n");

        wait(NULL);

        printf("\nChild process completed.\n");
        printf("Parent process completed.\n");
    }

    return 0;
}
```

Output :

```
jhanasri@LAPTOP-1RN1M8E3:~$ mkdir week6_process
jhanasri@LAPTOP-1RN1M8E3:~$ cd week6_process
jhanasri@LAPTOP-1RN1M8E3:~/week6_process$ pwd
/home/jhanasri/week6_process
jhanasri@LAPTOP-1RN1M8E3:~/week6_process$ nano process.c
jhanasri@LAPTOP-1RN1M8E3:~/week6_process$ gcc process.c -o process
jhanasri@LAPTOP-1RN1M8E3:~/week6_process$ ls
process  process.c
jhanasri@LAPTOP-1RN1M8E3:~/week6_process$ ./process
Parent process started.
Parent PID: 535

Parent is waiting for child process...

Child process created successfully.
Child PID: 536
Parent PID: 535

Executing ls command using execlp()...
total 20
-rwxr-xr-x 1 jhanasri jhanasri 16304 Sep 18 13:46 process
-rw-r--r-- 1 jhanasri jhanasri   968 Sep 18 13:45 process.c

Child process completed.
Parent process completed.
```

Result:

The child process was successfully created using `fork()` and the `ls` command was executed using `execlp()`. The parent process successfully waited for the child process, and the complete execution was verified through the displayed PIDs.

Observation:

The child process executed the required command successfully, while the parent process maintained synchronization using `wait()`. The program demonstrated successful process creation, program execution, and parent-child process management without errors.

